Assignment 1 solution

1. 【考点：操作系统的发展历程】关于计算机操作系统发展历程中的主要阶段及其核心特征，下列说法正确的是?

   A. 单道批处理系统每次只允许一个作业进入内存，因此其 CPU 和 I/O 设备的利用率始终能够保持在 100% 的最高水平
   B. 分时操作系统通过时间片轮转机制，使得多个终端用户能够"同时"与计算机进行交互，其响应时间长短与当前系统中的在线用户数量无关
   C. 实时操作系统的主要特征是能够及时响应外部事件的请求，并在严格规定的时间内完成对该事件的处理，其中硬实时系统绝对不允许错过任何截止期限
   D. 多道批处理系统允许多个程序同时驻留在单核 CPU 的内存中，且这些程序在物理时间上是完全并行执行的

   【答案】：C
   【解析】：
   A 选项错误：单道批处理系统由于内存中只有一个程序，当程序进行 I/O 操作时，CPU 必须空闲等待，因此资源利用率极低
   B 选项错误：分时系统的响应时间与时间片长度、用户数量以及系统调度开销均有密切关系，用户数越多响应越慢
   D 选项错误：在单核 CPU 中，多道程序的执行在宏观上是并发的，但在微观的绝对物理时间上是交替串行执行的，并非"完全并行"

2. 【考点：程序运行环境】现代计算机系统通过区分内核模式（管态）和用户模式（目态）来保护操作系统的核心数据。下列哪一条操作指令只能在内核模式下被执行?

   A. 将指定的通用数据寄存器的内容压入当前进程的用户态堆栈中
   B. 应用程序通过内部的控制流跳转指令，执行本进程代码段中的另一个普通函数
   C. 修改处理器状态字寄存器（PSW）中的中断允许标志位，以屏蔽外部硬件中断
   D. 用户进程读取当前分配给自身的一段虚拟内存空间中的只读数据

   【答案】：C
   【解析】：
   A、B、D 均为普通的算术逻辑、控制流和常规内存访问操作，属于非特权指令，可以在用户态下执行。C 选项涉及对底层硬件状态（中断开关）的修改，若允许用户随意屏蔽中断，将导致操作系统失去对 CPU 的控制权，因此修改 PSW 状态字必须且只能在内核态下作为特权指令执行

3. 【考点：程序运行环境】在操作系统运行期间，会发生引起 CPU 状态转换的各种事件。下列事件中，属于"异常"（内部异常/内部中断），且一定会导致 CPU 从用户态陷入内核态的是?

   A. 应用程序在进行算术运算时，执行了一条除数为 0 的除法指令
   B. 外部的系统时钟发生器产生了一个周期性的时钟滴答信号
   C. 磁盘控制器完成了操作系统之前下达的一批数据写入操作，发出完成信号
   D. 用户在控制台敲击键盘，输入了一个回车符

   【答案】：A
   【解析】：
   B、C、D 选项均来源于 CPU 执行指令外部的硬件设备（时钟、磁盘、键盘），属于异步的"外部中断"（Interrupt）。A 选项是 CPU 在执行当前指令的过程中产生的同步错误（Fault/Trap 类的异常），它源于指令本身，属于内部异常。一旦发生除零错误，硬件会自动触发异常处理机制，切换到内核态并由操作系统的异常处理程序接管

4. 【考点：程序运行环境】关于系统调用（System Call）的底层执行机制，下列描述正确的是?

A. 用户程序为了请求底层硬件服务，可以直接在自身代码中内嵌特权指令来发起系统调用请求
B. 应用程序通过执行特定的“陷入（Trap/软中断）”指令主动触发异常，从而将 CPU 控制权安全地交接给操作系统内核中的系统调用入口程序
C. 系统调用执行完毕并返回用户态后，操作系统必定会立即进行进程调度，将 CPU 分配给另一个优先级更高的就绪进程
D. 为了保证数据的绝对安全性，系统调用的所有参数只能通过寄存器进行传递，严禁使用内存区域进行参数传递

【答案】：B
【解析】：
A 选项错误：用户程序无权执行任何特权指令，系统调用是通过非特权指令（如 Trap、Int、Syscall 指令）引发异常来完成的
C 选项错误：系统调用返回时，多数情况下会直接返回给原先发起调用的进程继续执行，只有当系统中出现更高优先级进程或原进程主动阻塞时才一定会发生调度
D 选项错误：当参数数量较多时，寄存器不足以存放所有参数，操作系统通常允许通过将参数打包在内存块中，并将该内存块的首地址存入寄存器来进行参数传递

5. 【考点：程序运行环境】在将用户编写的源代码转化为可在内存中执行的进程时，需要经历编译、链接和装入等步骤。若某系统采用了“可重定位装入（静态重定位）”技术，下列说法正确的是？

   A. 目标模块在被装入内存时，装入程序会将其内部的所有逻辑地址一次性转换为绝对物理地址，且在整个运行期间物理地址不再改变
   B. 程序在运行期间可以被操作系统在内存中任意移动位置，并且移动后不需要修改程序内部的任何地址信息
   C. 这种装入技术必须依赖 CPU 硬件层面提供的重定位寄存器（基址寄存器）才能完成地址的映射
   D. 程序的各个独立目标模块，直到其实际执行到发生函数调用的代码行时，才会被链接器动态拼接在一起

   【答案】：A
   【解析】：
   A 选项是可重定位装入（静态重定位）的标准定义，由软件在装入时一次性完成逻辑地址到物理地址的修改
   B、C 选项描述的是“动态重定位（动态运行期装入）”的特征，它依赖硬件基址寄存器，且允许程序在内存中浮动
   D 选项描述的是“动态运行时链接（运行时动态链接）”技术，属于链接范畴而非静态装入方式

6. 【考点：程序运行环境】在一个经典的 Linux 进程内存布局（内存映像）中，若程序员编写的 C 语言程序包含了全局变量、局部变量以及动态内存分配。关于这些数据所在段的特征，下列说法错误的是？

   A. 源代码中未初始化的全局变量会被编译器分配到 `.bss` 段，该段在磁盘可执行文件中不实际存储大块的零数据，仅记录需要清零的内存大小
   B. 进程运行时，通过 `malloc` 函数动态申请的内存空间位于堆（Heap）区，堆区的虚拟地址通常由低地址向高地址方向增长
   C. 函数调用过程中的参数传递、局部变量以及函数的返回地址，均会被操作系统分配并保存在代码段（`.text` 段）中
   D. 程序的机器指令本身存放在只读的代码段（`.text` 段）中，以防止指令在运行期间被恶意篡改或因程序 Bug 被意外覆盖

   【答案】：C
   【解析】：
   C 选项错误：函数调用时的参数、局部变量以及返回地址是动态压入和弹出的，它们存放在用户的栈（Stack）区中，而不是代码段中。代码段是只读的，主要用于存放执行的机器指令。其他选项均正确描述了 C 程序内存布局的特征

7. 【考点：操作系统结构】相比于将所有核心功能代码打包在一个大且复杂的执行空间内的宏内核（Monolithic Kernel），现代操作系统中采用微内核（Microkernel）架构的主要设计意图及特点是?

A. 通过将绝大多数操作系统服务（如文件系统、网络协议）保留在内核态执行，微内核显著降低了系统的运行内存占用
B. 微内核仅将最底层的核心功能（如进程间通信机制、最基础的硬件管理）保留在内核中，而将其他服务剥离为独立运行在用户态的用户进程
C. 由于微内核系统中不同模块都在用户态直接调用彼此的函数，因此它彻底消除了进程间通信（IPC）造成的性能损耗
D. 当微内核结构中的一个非核心服务（如显卡驱动进程）发生崩溃时，必然会导致整个操作系统内核随之崩溃（Kernel Panic）

【答案】：B
【解析】：
A 选项错误：微内核正是将大量服务移出了内核态
C 选项错误：用户态的服务模块之间由于处于不同的地址空间，不能直接进行函数调用，必须通过微内核提供的基于消息传递的 IPC 机制进行通信，这正是微内核性能开销较大的根本原因
D 选项错误：微内核的优势在于极高的可靠性。由于非核心服务运行在独立的用户态进程中，如果显卡驱动崩溃，仅该进程会死亡，不会波及内核本身及其他服务

8. 【考点：操作系统引导】对于传统基于 BIOS 的 x86 架构计算机，当接通电源并顺利完成通电自检（POST）后，操作系统的正常引导序列通常是?

A. 读取并执行主引导记录（MBR）代码 → 执行活动分区的引导扇区代码（PBR） → 加载并执行引导装入程序（Bootloader 主体） → 将操作系统内核加载进内存并初始化
B. 优先将操作系统内核加载进内存 → 读取并执行主引导记录（MBR）代码 → 执行活动分区的引导扇区代码（PBR） → 执行引导装入程序（Bootloader 主体）
C. 执行引导装入程序（Bootloader 主体） → 将操作系统内核加载进内存并初始化 → 读取并执行主引导记录（MBR）代码 → 执行活动分区的引导扇区代码（PBR）
D. 执行活动分区的引导扇区代码（PBR） → 将操作系统内核加载进内存并初始化 → 读取并执行主引导记录（MBR）代码 → 执行引导装入程序（Bootloader 主体）

【答案】：A
【解析】：
计算机启动是一个从固件逐步加载复杂软件的链式过程：BIOS 验证硬件后，首先读取磁盘第 0 面 0 道 1 扇区的主引导记录（MBR）；MBR 中的代码负责找到拥有系统文件的活动分区，并加载该分区第一个扇区（PBR）；PBR 再去文件系统中找到并加载更强大的 Bootloader（如 GRUB 的第二阶段）；最后由 Bootloader 将庞大的操作系统内核（如 vmlinuz）读入内存并移交控制权

9. 【考点：虚拟机】在虚拟化技术中，虚拟机监控程序（Hypervisor/VMM）通常被分为第一类（裸机型）和第二类（宿主型）。下列关于这两类 VMM 的描述正确的是?

A. 第一类和第二类 VMM 在底层实现上完全相同，都必须依赖一个已经完整安装并运行的宿主操作系统（Host OS）来提供 I/O 驱动支持
B. 第一类 VMM 直接安装在裸金属物理硬件上，全面接管 CPU 和内存的管理调度权，在企业级数据中心服务器中应用最为广泛（如 VMware ESXi）
C. 第二类 VMM 本身被视为硬件层面的固件，它可以允许客户机操作系统（Guest OS）完全绕过 VMM 直接执行所有的特权指令
D. 在第二类 VMM 环境下运行的客户机系统（Guest OS），其执行速度和 I/O 性能通常优于第一类 VMM 环境下的客户机系统

【答案】：B
【解析】：
A 选项错误：第一类 VMM（裸机型）不需要宿主操作系统，它本身就是一个精简版的操作系统内核
C 选项错误：第二类 VMM（如 VMware Workstation）是运行在宿主操作系统上的应用程序，

Guest OS 的特权指令必须被 VMM 拦截和模拟，不可能直接穿透
D 选项错误：第二类 VMM 因为多了一层宿主操作系统的开销（Guest -> VMM -> Host OS -> Hardware），其性能通常低于直接与硬件交互的第一类 VMM

10. 【考点：系统运行环境的状态转换】某系统正在执行用户代码 `int fd = open("/test.txt", O_RDONLY);`。当应用程序执行至对应的库函数内部触发底层系统调用时，关于该过程引起的软硬件协同工作机制，下列说法必定错误的是？

    A. 若应用程序在没有使用系统调用的情况下，企图通过指针越界直接读取内存中的内核态堆栈数据，硬件会静默忽略该读取指令并返回全零数据
    B. 执行系统调用的陷阱指令（Trap）会引发软中断，此时硬件机制会自动将 CPU 中的当前状态字（PSW）以及程序计数器（PC）压入内核栈中进行保护
    C. 在 CPU 陷入内核态后，操作系统会根据寄存器中传入的系统调用号，在系统调用入口表中查找对应的处理函数（如 `sys_open`）的首地址
    D. 当该系统调用执行完毕后，操作系统内核会通过一条特权级别的返回指令（如 IRET），将 CPU 的运行模式重新恢复为用户态

    【答案】： A
    【解析】：
    A 选项说法必定错误。在现代包含内存管理单元（MMU）的计算机系统中，用户态程序如果试图访问属于内核空间或未授权的内存地址，硬件的内存保护机制会立即触发一次严重的“段错误”或“一般保护性异常（General Protection Fault）”，导致操作系统强行终止该进程，而不是“静默忽略并返回全零”。B、C、D 均是执行系统调用陷入、分发处理以及返回机制的标准过程

11. 【考点：操作系统的发展历程】在量化金融的高频交易（HFT）系统或航空航天飞控系统中，对计算机响应时间的极度苛求推动了操作系统的选型。关于不同发展阶段的操作系统的基本特征与工程适用场景，下列说法完全正确的是（ ）

    A. 多道批处理系统（Multiprogramming Batch System）通过作业调度算法最大化了 CPU 和 I/O 设备的并行工作能力，因此它是目前高并发实时交易系统最理想的基础架构
    B. 现代分时操作系统（Time-sharing System，如标准 Linux）通过时间片轮转机制，能够绝对保证高频交易程序在 1 微秒（μs）内获得确定性的 CPU 响应
    C. 实时操作系统（RTOS）的主要设计目标是追求极高的系统吞吐量和整体资源的最高利用率
    D. 在硬实时系统（Hard Real-time System）中，哪怕计算结果是完全正确的，只要该结果未在极其严格的绝对截止期限（Deadline）内交付，系统就会被认为发生了致命的灾难性故障

    【答案】： D
    【解析】：
    D 选项完全正确：硬实时系统（如飞控、汽车防抱死制动、高频交易核心）的根本特征是“时限的绝对严格性”。计算的正确性不仅取决于逻辑结果是否正确，还取决于结果产生的时间。错过了绝对截止期限（Deadline），即使数据正确，也会导致灾难性后果
    A 选项错误：多道批处理系统的核心目标是“提高吞吐量和资源利用率”，它的致命弱点是毫无交互性且响应时间不可预测，绝对不能用于实时交易系统
    B 选项错误：分时操作系统的核心目标是“公平”和“交互性”。由于时间片大小（通常为毫秒级）以及复杂的调度策略，分时系统只能提供软实时或无实时保证，绝对无法保证微秒级的“确定性延迟”
    C 选项错误：实时操作系统的核心目标是“及时性（Timeliness）”和“可靠性”，为了保证关键任务的绝对准时，RTOS 常常会牺牲系统的整体吞吐量和资源利用率

12. 【考点：CPU运行模式】在传统 Linux 内核中，高达千万级并发的网络数据包收发会因频繁的硬件中断而耗尽 CPU 资源。为了突破这一瓶颈，现代工程中广泛应用 DPDK（Data Plane Development Kit）等“内核旁路（Kernel Bypass）”技术。关于操作系统底层的中断、异常机制与 CPU 运行模式，下列说法完全正确的是（ ）

    A. DPDK 的核心原理是将网卡驱动移至用户态，并采用 CPU 轮询（Polling）机制读取网卡数据，从而彻底消除了高频外部硬件中断带来的内核态/用户态上下文切换开销
    B. 外部硬件中断（如网卡接收到数据包）属于同步异常，它必须等待 CPU 当前正在执行的指令运行到特定微操作阶段时才能被触发

C. 缺页异常（Page Fault）属于故障（Fault）类内部异常，当操作系统的缺页中断处理程序将页面调入内存并返回后，CPU 会跳过引发异常的指令，直接执行下一条指令
D. 只要应用程序在用户态下执行，就绝对不可能引发内部异常，所有的内部异常（如除零错误、非法操作码）都必须且只能在内核态下才能被引发

【答案】： A
【解析】：
A 选项完全正确：传统收包流程中，网卡每收到数据包就会向 CPU 发送外部中断，强迫 CPU 从用户态切换到内核态执行中断服务程序（ISR），开销极大。DPDK 等技术让用户态应用程序直接接管物理网卡内存，使用 `while(1)` 轮询直接读取数据，彻底避开了中断处理和特权级切换，实现了极限性能
B 选项错误：外部硬件中断是典型的异步事件，它与 CPU 当前正在执行的指令毫无关系。CPU 会在每条指令执行周期的末尾（中断采样阶段）去检查是否有外部中断请求
C 选项错误：缺页异常属于“故障（Fault）”。Fault 的显著特征是：当异常处理程序（调页完成）返回时，必须重新执行（Restart）引发该异常的那条访存指令，而不是跳过它执行下一条指令
D 选项错误：内部异常（如除零、越权访存、非法指令）恰恰最常由用户态程序的不当行为引发。当用户态触发这些硬件异常时，CPU 硬件会自动捕获并陷入内核态进行处理（如发送 `SIGFPE` 信号杀死该用户进程）

13. 【考点：程序的链接与装入】在云原生架构中，一台物理宿主机上可能同时运行着成百上千个微服务容器实例。为了大幅度降低物理内存的消耗，现代操作系统广泛使用了动态链接（Dynamic Linking）与共享库（Shared Libraries，如 `.so` 文件）。关于程序运行时内存映像与链接机制，下列说法完全正确的是（ ）

A. 静态链接（Static Linking）会在编译链接时将所有需要的库函数代码拷贝到最终的可执行文件中。因此，多个运行的同类进程可以在物理内存中共享这些静态库的代码段，从而节省内存
B. 在基于动态链接的系统中，`.so` 共享库的代码段（Text Segment）在宿主机的物理内存中只需保留一份副本，但每个调用该库的用户进程在自己的虚拟地址空间中都有各自独立的数据段（Data/BSS）映射
C. C语言程序装入内存后，未初始化的全局变量存放在数据段（`.data`），而程序在运行期间通过 `malloc` 动态分配的内存块则从栈（Stack）区向低地址方向不断分配
D. 地址无关代码（PIC）机制使得动态链接库在装入内存时，必须由操作系统的装入程序动态修改库中代码段的跳转指令相对地址，才能保证其在不同进程虚拟地址空间中的正确执行

【答案】： B
【解析】：
B 选项完全正确：这就是共享库（动态链接库）节省内存的核心原理。动态链接库的只读代码段（`.text`）在物理内存中只有一份，通过操作系统的页表映射到各个进程的虚拟地址空间中实现物理页共享。但是，库中的全局变量等数据（`.data` / `.bss`）必须是每个进程私有的（写时复制），以免互相干扰
A 选项错误：静态链接由于将库代码直接打包进了各自的可执行文件中，操作系统无法将其识别为公共的共享对象，因此多个相同程序的进程在内存中会产生多个完全相同的代码副本，极其浪费物理内存
C 选项错误：未初始化的全局变量存放在 `.bss` 段，而非 `.data` 段（`.data` 存放已初始化的非零全局变量）。此外，`malloc` 动态分配的内存存在于堆（Heap）区，并向高地址方向增长；栈（Stack）区主要用于函数局部变量，向低地址方向增长
D 选项错误：地址无关代码（Position Independent Code, PIC）的核心优势恰恰是不需要修改代码段（Text Segment）的内容。它通过引入全局偏移表（GOT）和过程链接表（PLT）在数据段中进行间接寻址，从而保证了代码段的绝对只读属性，使得多个进程可以共享同一个未被修改过的物理代码页

14. 【考点：操作系统结构】智能网联汽车（如自动驾驶控制域）对操作系统的安全性和容错性有着极为苛刻的要求，一旦系统内核崩溃将直接威胁生命安全。在宏内核（Monolithic）、微内核（Microkernel）等架构选型中，业界常采用 QNX 等微内核架构。下列关于操作系统结构特征的论述，完全正确的是（ ）

A. 宏内核架构将所有底层系统服务打包在内核空间，虽然其内部函数调用通信开销极小，但如果某个非核心组件（如第三方车载娱乐驱动程序）发生内存越权访问，将直接导致整个系统内核崩溃（Kernel Panic）
B. 微内核架构由于将文件系统、网络协议栈等移至用户态运行，消除了频繁的系统调用开销，因此其在自动驾驶系统中的整体调度延迟和 IPC 通信效率显著优于宏内核
C. 模块化（Modular）结构与分层（Layered）结构的本质区别在于：模块化结构严格限制了系统内部模块之间的单向依赖和调用关系，绝对禁止任何形式的跨层或逆向调用
D. 外核（Exokernel）架构的核心设计思想是为用户进程提供比宏内核更加丰富、更加高度抽象的硬件接口（如提供极其复杂的分布式文件系统和数据库操作 API）

【答案】： A

【解析】：
A 选项完全正确：宏内核（如 Linux）的所有内核级代码共享同一个巨大的地址空间（Ring 0）。这意味着任何一个细微的 Bug（比如一个不合格的第三方设备驱动发生空指针解引用）都会直接破坏整个内核的数据结构，导致系统整体宕机。这种“一损俱损”的特性是宏内核在安全性上的致命软肋
B 选项错误：微内核虽然将服务移到用户态从而提高了安全性与容错性（某服务崩溃只需重启该用户进程即可），但进程间的通信必须依靠微内核的 IPC（进程间通信）机制进行消息传递，这会带来海量的“用户态-内核态-用户态”上下文切换开销。微内核的通信效率远低于宏内核的直接函数调用
C 选项错误：描述完全反了。分层（Layered）结构才具有严格的单向依赖规则，只允许上层调用下层，禁止越级调用；而模块化（Modular）结构非常灵活，各个模块之间只要接口匹配，就可以相互调用，没有严格的层级限制（这也是现代 Linux 采用的方法）
D 选项错误：外核（Exokernel）的设计哲学是“取消抽象，直接暴露物理资源”。它不仅不提供高度抽象的接口，反而将资源的抽象和管理任务全部下放给用户态的系统库（LibOS）去完成，以便应用程序能够为了极致性能去自由定制物理资源的分配规则

15. 【考点：操作系统引导】在公有云平台（如 AWS、阿里云）中，物理服务器通过虚拟机管理程序（Hypervisor/VMM）被划分为多个隔离的云主机（VM）租给不同的客户。关于虚拟机的分类、硬件辅助虚拟化技术以及客户操作系统的引导过程，下列描述完全正确的是（ ）

A. 无论是一类（Type-1 裸机型）还是二类（Type-2 宿主型）Hypervisor，虚拟机内部的客户操作系统（Guest OS）在开机引导时，都是直接读取物理宿主机主板上的 BIOS/UEFI 固件来进行加电自检的
B. 虚拟机中的 Guest OS 在执行如修改页表基址等敏感特权指令时，如果采用纯软件模拟（Trap-and-Emulate），底层硬件会直接将该指令自动降级为无害的用户态普通指令并继续执行，以防止宿主机系统被破坏
C. 在标准 PC 体系中，主引导记录（MBR）及其包含的初始引导加载程序（Bootloader）被出厂固化在物理主板的 ROM 只读芯片中，即使整个磁盘损坏也不会丢失
D. 硬件辅助虚拟化（如 Intel VT-x）通过在 CPU 中新增指令运行模式（如 VMX Root / Non-Root 模式），使得 Guest OS 内核可以无需任何修改地运行在 Non-Root 的最高特权级（Ring 0）下，大幅降低了系统因陷入 VMM 而造成的上下文切换损耗

【答案】： D
【解析】：
D 选项完全正确：传统的软件虚拟化需要复杂的陷入-模拟机制，性能较差。硬件辅助虚拟化（Intel VT-x/AMD-V）在硬件层面增加了两套平行世界：VMX Root（Hypervisor 运行于此的 Ring 0）和 VMX Non-Root（Guest OS 运行于此的 Ring 0）。这样 Guest OS 觉得自己在 Ring 0 拥有全部特权，直接执行大部分指令不再频繁触发 Trap，极大地提升了虚拟机的执行性能
A 选项错误：虚拟机中的硬件环境是虚拟出来的。Guest OS 启动时，读取的是 Hypervisor 为它提供的虚拟 BIOS/UEFI 固件代码（通常是一个存在于宿主机内存或磁盘上的镜像文件），绝对不可能直接去碰物理主板上的真实固件
B 选项错误：在 Trap-and-Emulate 机制下，当运行在虚拟环境用户态（如 Ring 1 或 Ring 3）的 Guest OS 尝试执行敏感指令时，硬件会触发特权级违例异常（Trap），强制陷入到 VMM 中。由

VMM 在宿主机的真实 Ring 0 中判断意图并通过软件逻辑模拟该指令的执行效果，然后再将结果返回给 Guest OS，绝不是“自动降级继续执行”

C 选项错误：主板 ROM 中固化的是 BIOS/UEFI 代码。而主引导记录（MBR）位于系统启动磁盘的“第 0 柱面、第 0 磁头、第 1 扇区”（即磁盘的最开头），它是存储在磁盘物理介质上的。如果磁盘被清空或损坏，MBR 就会丢失

16. **33．A**

当中断或异常发生时，CPU既可能处于内核态，又可能处于用户态，具体取决于当时CPU正在处理的任务，选项A错误。不同的系统调用对应不同的内核服务例程，选项B正确。在中断响应阶段，若CPU处于用户态，则需要切换到内核态，因此在中断处理阶段，CPU一定处于内核态，选项C正确。设备种类繁多，计算机不可能事先准备好所有设备对应的中断服务例程（实际上属于设备驱动程序），因此当系统添加新类型的设备时，需要注册相应的中断服务例程。

17. **22．C**

虚拟化技术通过软件抽象底层硬件资源，使多个相互隔离的虚拟机能够在同一台物理计算机上运行。操作系统可作为客户系统运行在虚拟机中，依托虚拟硬件正常运行；同时，在单台物理主机上构建多个虚拟机正是虚拟化的核心能力。虚拟机监控程序（VMM）作为虚拟化的核心组件，必须运行在**高于操作系统**的特权级上，才能有效拦截和模拟敏感指令、管理物理资源并维持虚拟机间的隔离，因此其特权级不可能与操作系统的相同。此外，结合仿真或动态二进制翻译等机制，虚拟化方案也能在一种指令集架构的平台上模拟运行另一种架构的虚拟机，进而支持跨架构场景。综上，选项 C 的表述不符合虚拟化的基本原理。

18. 【考点：CPU 运行模式】关于程序运行环境、CPU 运行模式切换以及中断/异常处理机制，下列说法中正确的是（ ）

A. 陷入指令（Trap/Syscall 指令）属于特权指令，只能在内核态（管态）下执行

B. 发生缺页异常（Fault）时，中断处理程序执行完毕后，CPU 会自动返回至引发缺页指令的下一条指令继续执行

C. 进程从用户态切换到内核态时，CPU 硬件会自动将用户栈切换为内核栈，但该过程并不一定会引发进程上下文（Context）的切换

D. 外中断（如时钟中断、I/O 中断）的产生与 CPU 当前执行的指令严格同步，因此 CPU 会在指令执行的中途立即中断并响应

【答案】： C

【解析】：

A 项错误：陷入指令（如 `int 0x80` 或 `syscall`）是用户程序主动申请系统服务时调用的指令，必须在用户态下执行；若其为特权指令，则用户态下无法触发，引发矛盾

B 项错误：缺页属于故障（Fault）。故障处理完成后，CPU 会返回并重新执行引发故障的那条指令（重新读取内存数据），而不是下一条指令；只有中断（Interrupt，如外中断）或陷入（Trap）处理完后才返回下一条指令

C 项正确：当发生模式切换（如系统调用或时钟中断）时，CPU 硬件会自动从 TSS（任务状态段）中读取该进程的内核栈指针并完成栈切换。若系统调用处理完后无需重新调度（例如仅读取系统时间），则原进程继续执行，只发生了模式切换（Mode Switch），并未发生进程上下文（Context Switch）切换

D 项错误：外中断是由 CPU 外部硬件发出的，与 CPU 执行的指令异步。CPU 通常在每条指令执行周期的末尾（中断采样阶段）检查并响应外中断，不会在指令执行中途直接打断

19. 【考点：操作系统结构（微内核、外核）】操作系统内核架构与虚拟机（VMM）技术密切相关。下列关于操作系统结构（微内核、外核）及虚拟机系统的叙述中，错误的是（ ）

A. 微内核架构将传统内核中的文件系统、设备驱动等移至用户态，虽然提升了系统的可扩展性与安全性，但增加了内核态与用户态之间的切换开销

B. 外核（Exokernel）架构的核心思想是尽量减少内核对硬件资源的抽象，将资源分配与资源管理（如虚拟内存映射）分离，直接向用户应用暴露硬件接口

C. 第一类（Type-1，如 Xen、VMware ESXi）虚拟机管理程序直接运行在硬件之上，其客户操作系统（Guest OS）运行在绝对最高特权级（内核态）

D. 第二类（Type-2，如 VirtualBox）虚拟机管理程序作为宿主操作系统（Host OS）的一个应用程序运行，依赖宿主 OS 完成底层硬件资源调度

【答案】： C

【解析】：

A 项正确：微内核仅保留最核心的服务（如进程间通信 IPC、进程调度、基本内存管理）。服务模块间通信需要频繁通过 IPC 穿越用户态与内核态，增加了系统开销

B 项正确：外核（Exokernel）取消了传统操作系统对硬件的统一抽象（如把磁盘抽象为文件系统），外核仅负责分配与保护硬件资源，具体管理交给用户态库（LibOS），从而提供更高的灵活性与性能

C 项错误：在第一类 Hypervisor 中，VMM（或 Hypervisor 自身）直接运行在最高特权级（如 x86 的 Ring 0 / Root 模式）。为了防止 Guest OS 破坏 VMM 的控制，Guest OS 必须运行在受限的特权级（如 Ring 1 或 Non-root 模式），其执行特权指令时会被 VMM 截获并剥夺/模拟。因此 Guest OS 不处于绝对最高特权级

D 项正确：第二类 VMM 依赖 Host OS，其虚拟化实现需要将 Guest OS 的硬件指令映射为 Host OS 的系统调用或进程操作

20. 【考点：程序的链接与装入】关于程序的链接与装入机制以及程序运行时的内存映像与地址空间，下列说法中正确的是（ ）

A. 静态重定位（Static Relocation）是在装入程序时完成逻辑地址到物理地址的转换，因此程序在运行期间可以随时在内存中移动

B. 动态运行时装入（Dynamic Run-time Loading）要求程序装入内存时，必须将所有指令和数据的逻辑地址一次性转换为物理地址

C. C 语言程序中的未初始化全局变量在编译后会被放置在 `.data` 节中，并直接占用可执行文件（如 ELF）的大量磁盘空间

D. 装入时动态链接（Load-time Dynamic Linking）发生在程序装入内存时，边装入边链接；而运行时动态链接（Run-time Dynamic Linking）可实现目标模块直到被实际调用时才装入并链接

【答案】： D

【解析】：

A 项错误：静态重定位是在程序装入内存时一次性将逻辑地址修改为物理地址，装入后地址即被固定，运行期间不能在内存中移动；只有动态重定位（依靠重定位寄存器）才允许程序在内存中动态移动

B 项错误：动态运行时装入（配合动态重定位）是将地址转换推迟到指令真正被执行时才进行，而不是装入时一次性转换

C 项错误：未初始化的全局变量和静态变量放置在 `.bss` 节中。`.bss` 节在 ELF 磁盘文件中不占用实际的数据存储空间（仅记录所需空间大小），只有在程序加载至内存创建内存映像时，操作系统才为其分配物理内存并清零

D 项正确：精准区分了装入时动态链接（Load-time）与运行时动态链接（Run-time，如 Windows 下的 `LoadLibrary` / `GetProcAddress` 或 Linux 下的 `dlopen`）

21. 【考点：操作系统引导】操作系统引导（Booting）涉及计算机硬件、固件与操作系统的协同。下列关于传统基于 BIOS 的 OS 引导过程与系统初始化的叙述中，错误的是（ ）

A. ROM 中的 BIOS 程序在计算机通电后最先执行，此时 CPU 处于实模式（相当于具备最高访问权限）

B. 主引导记录（MBR）位于磁盘的 0 头 0 柱面 1 扇区，包含主引导程序和分区表信息，其总大小通常为 512 字节

C. Bootloader（如 GRUB）的主要任务是将内核映像加载到内存中，并在跳转至内核入口点前建立初步的内核栈与模式切换

D. 中断向量表（IVT 或 IDT）由 BIOS 在 ROM 中硬编码固化，操作系统启动后无法修改其内容

【答案】： D

【解析】：

A 项正确：通电后 CPU 硬件逻辑将 PC 寄存器重置指向 ROM BIOS 的入口地址。此时 CPU 处于实模式，可以无限制访问所有物理内存和外设

B 项正确：MBR 固定存储于 0/0/1 扇区，占用 512 字节（446 字节引导代码 + 64 字节分区表 + 2 字节结束标记 `0x55AA`）

C 项正确：Bootloader 负责从磁盘读取内核文件到内存，设置 CPU 运行模式（如从 16 位实模式切换到 32/64 位保护/长模式），设置页表与栈指针，最后跳转至内核入口

D 项错误：BIOS 仅在开机初期建立实模式下的简易中断向量表。操作系统启动后（尤其进入保护模式后），会在内存中重新构建自己的中断描述符表（IDT），并通过 CPU 特权指令（如 `LIDT`）将 IDT 的基地址装入 CPU 寄存器，从而完全接管系统的所有中断与异常处理

22. 【考点：中断/异常响应流程】当用户程序执行过程中触发中断或异常时，CPU 与操作系统需协同完成现场保存与服务响应。下列关于该过程中硬件（硬件隐指令）与软件（中断处理程序）分工的说法中，正确的是（ ）

A. 保存断点（程序计数器 PC）和程序状态字（PSW）的工作必须由中断处理程序通过执行软件指令显式完成

B. 从用户栈切换到内核栈、以及将原用户栈指针（如 SS 和 ESP）压入内核栈的操作是由 CPU 硬件自动完成的

C. 中断向量（Interrupt Vector）保存的是中断处理程序的入口地址，硬件可根据中断向量直接跳转至内核函数执行

D. 执行陷入指令进行系统调用时，CPU 硬件会自动将用户程序传递的所有参数依次压入内核栈中

【答案】： B

【解析】：

A 项错误：保存断点（PC）和程序状态字（PSW）必须由 CPU 硬件（硬件隐指令） 自动压栈保存。因为在执行软件指令之前，PC 必须被改变以跳转到中断服务程序，如果由软件保存，原本的断点值已经被覆盖而无法恢复

B 项正确：当发生特权级转换（用户态 → 内核态）时，CPU 硬件会自动从 TSS 中获取当前进程的内核栈指针，将栈切换至内核栈，并自动将用户态的 SS 和 ESP（用户栈指针）压入内核栈，以保证内核栈上下文的完整性

C 项错误：中断向量（Interrupt Vector）是中断/异常的编号/索引（例如 `0x80`），而不是入口地址本身；硬件根据中断向量查中断向量表（或中断描述符表 IDT），才能获取对应的中断服务程序入口地址

D 项错误：系统调用的参数传递通常由软件（用户程序和内核函数约定）通过通用寄存器（如 x86-64 的 `rdi`, `rsi` 等）或显式内存传递；硬件在触发陷入指令时仅完成模式切换和硬件上下文（PC、PSW、栈指针）的压栈，绝不会自动解析并压入用户层面的业务参数

23. 【考点：中断与异常处理】关于 CPU 运行模式、中断与异常的微观响应过程及硬件隐指令的功能，下列说法中正确的是（ ）

A. 当发生缺页故障（Fault）时，CPU 在执行硬件隐指令期间将断点（下一条指令的地址）压入内核栈

B. 系统调用通过执行陷入指令（Trap）实现，CPU 切换至内核态后，硬件隐指令会自动将用户栈指针及系统调用参数全部压入内核栈

C. 外中断响应发生在一条指令执行周期的末尾，硬件隐指令会自动关中断、保存 PSW 与 PC，并根据中断向量查找 IDT 跳转至服务程序

D. 若在执行内核态代码时触发硬件故障（Abort），CPU 将直接切换回用户栈并调用用户态信号处理函数

【答案】： C

【解析】：

A 项错误：缺页属于故障（Fault）。故障处理完成后需要重新执行引发异常的那条指令，因此硬件隐指令压入内核栈的断点是当前引发缺页的指令地址，而不是下一条指令地址

B 项错误：发生特权级切换时，硬件隐指令会自动将 SS、ESP、EFLAGS/PSW、CS、EIP/PC 压入

内核栈，但系统调用传递的业务参数是由软件（用户程序通过寄存器或压栈）传递的，硬件隐指令绝不会自动解析并压入用户参数

C 项正确：外中断与指令执行异步，CPU 在每条指令执行周期的末尾（中断采样阶段）检测外中断。响应中断时，硬件隐指令会自动关中断（禁止重入）、保存 PSW 和 PC 到内核栈，并根据中断向量查中断描述符表（IDT）获取服务例程入口

D 项错误：内核态下触发终止（Abort，如硬件校验错误）属于严重致命错误，系统通常直接进入内核 Panic/宕机处理，不可能切回用户栈执行用户态函数

24. 【考点：操作系统结构】操作系统架构的设计直接影响系统的性能与扩展性。下列关于操作系统结构（微内核、外核、宏内核及模块化结构）的叙述中，正确的是（ ）

A. 微内核架构将文件系统与设备驱动置于用户态，应用进程请求读取磁盘文件时，整个过程只需发生 2 次用户态与内核态的模式切换

B. 外核（Exokernel）架构的核心思想是尽量减少内核对硬件资源的抽象，将资源分配与资源管理（如虚拟内存映射）解耦，直接向用户应用暴露物理资源接口

C. 在宏内核结构中，所有系统服务均运行在内核态，因此执行系统调用时只需进行寄存器上下文保存，不涉及任何栈指针的切换

D. 模块化内核（Modular Kernel）在运行时动态加载驱动模块，此过程会将驱动加载至专门的微内核特权环（Ring 1）中运行

【答案】： B

【解析】：

A 项错误：在微内核中，应用进程发请求到内核（1次切换），内核通过 IPC 转给用户态文件系统（1次切换），文件系统处理完发给内核（1次切换），内核转给磁盘驱动（1次切换）……一次完整的文件 IPC 读写通常包含 4 次或更多 的用户态与内核态模式切换

B 项正确：外核（Exokernel）取消了传统 OS 对硬件的统一抽象（如将磁盘抽象为文件），外核仅负责硬件资源的保护与安全分配，具体的资源管理和抽象交由用户态的库操作系统（LibOS）实现，实现了分配与管理的分离

C 项错误：即使在宏内核下，系统调用触发特权级切换（用户态 → 内核态）时，CPU 硬件隐指令也会自动将用户栈指针切换为 TSS 中记录的内核栈指针，必然涉及栈指针的切换

D 项错误：模块化内核仍属于宏内核范畴，动态加载的驱动模块依然运行在真正的内核态（如 x86 的 Ring 0），不存在所谓的“微内核特权环”

25. 【考点：虚拟机管理程序分类】虚拟化技术依赖于硬件架构与操作系统内核的紧密配合。下列关于虚拟机管理程序（Hypervisor/VMM）及虚拟化机制的说法中，正确的是（ ）

A. 只要某条指令是特权指令，它就必然属于敏感指令；反之，非特权指令绝不可能是敏感指令

B. 第一类 VMM（Type-1）直接运行在裸机硬件上，其客户操作系统（Guest OS）运行在绝对最高特权级（Ring 0）

C. 在传统纯软件虚拟化中，若存在“非特权敏感指令”，Guest OS 在用户态下执行该指令时不会触发异常陷阱（Trap），从而导致虚拟化漏洞

D. 第二类 VMM（Type-2）中的 Guest OS 发生系统调用时，直接由硬件隐指令跳转至 Guest OS 内核，无需宿主操作系统（Host OS）参与

【答案】： C

【解析】：

A 项错误：敏感指令是指企图修改硬件配置或行为依赖于底层特权状态的指令。某些指令在早期 x86 上是非特权指令（在用户态下执行不报错、不触发 Trap），但它们又是敏感指令（如读取 CS 寄存器获取当前特权级），这类指令被称为“非特权敏感指令”

B 项错误：在 Type-1 VMM 中，VMM 本身占据最高特权级（Ring 0 / VMX Root），Guest OS 被降级运行在受限特权级（如 Ring 1 或 VMX Non-root），不能直接运行在真正的 Ring 0

C 项正确：传统的“陷入-模拟”（Trap-and-Emulate）虚拟化要求所有敏感指令都是特权指令。如果存在“非特权敏感指令”，Guest OS 执行它时硬件不会产生 Trap，导致 VMM 无法捕获并模拟它（Guest OS 拿到了错误的硬件状态），这就是 x86 早期著名的虚拟化缺陷（通过二进制代码翻译或硬件辅助虚拟化 Intel VT-x 解决）

D 项错误：Type-2 VMM 本身是 Host OS 上的一款应用程序，Guest OS 的所有特权操作和系统调用都必须经过 VMM 截获，并转换为 Host OS 的系统调用或进程请求，无法绕过 Host OS

26. 【考点：程序的链接与装入】程序的编译、链接、装入以及运行时内存映像是操作系统运行程序的基础。下列关于该过程的叙述中，正确的是（ ）

A. 静态重定位（Static Relocation）在装入程序时完成逻辑地址到物理地址的转换，因此程序在运行期间可以根据需要动态移动位置
B. C 语言程序中未初始化的全局变量存放在 ELF 可执行文件的 `.bss` 节中，因此 `.bss` 节在磁盘文件中占据与这些变量实际大小相等的存储空间
C. 装入时动态链接（Load-time Dynamic Linking）发生在程序装入内存时；而运行时动态链接（Run-time Dynamic Linking）直到目标模块被实际函数调用时才进行装入与目标代码地址绑定
D. 采用动态运行时装入（Dynamic Run-time Loading）的系统，代码段（`.text`）在内存中移动后，操作系统必须显式修改代码段中所有包含绝对地址的指令

【答案】： C
【解析】：
A 项错误：静态重定位是在装入内存时一次性将逻辑地址修改为物理地址，装入后物理地址固定，运行期间绝对不能在内存中移动。只有动态重定位（依靠重定位寄存器）才允许程序动态移动
B 项错误：未初始化的全局变量和静态变量放置在 `.bss` 节中。`.bss` 节在 ELF 磁盘文件中不占用实际的数据存储空间（仅记录所需空间大小的元数据）；只有在装入内存建立内存映像时，操作系统才为其分配清零的物理内存
C 项正确：准确区分了装入时动态链接（边装入边链接）与运行时动态链接（如 `dlopen` / `LoadLibrary`，只有执行到特定代码时才加载并绑定地址）
D 项错误：动态运行时装入依靠硬件重定位寄存器（基址寄存器），在 CPU 执行取指/访存时 自动将逻辑地址与基址相加，无需修改代码段内的逻辑地址

27. 【考点：操作系统引导】计算机系统的引导（Booting）涉及硬件、固件与操作系统内核的交接。下列关于基于传统 BIOS 的引导过程及系统初始化的说法中，错误的是（ ）

A. 计算机加电后，CPU 硬件逻辑将 PC 重置并指向 ROM BIOS 入口地址，此时 CPU 运行在实模式下
B. 主引导记录（MBR）固定存在于磁盘的 0 头 0 柱面 1 扇区，其内部包含主引导程序和 64 字节的分区表
C. 在 BIOS 引导流程中，操作系统内核在 16 位实模式下直接初始化页表并开启 MMU 分页机制，随后切换至 32 位保护模式
D. 操作系统引导并接管系统后，会将新的中断描述符表（IDT）基地址装入 CPU 的 `IDTR` 寄存器，从而覆盖 BIOS 初始化的实模式中断向量表（IVT）

【答案】： C
【解析】：
A 项正确：通电瞬间 CPU 处于实模式，硬件逻辑强制将 PC/CS:IP 指向 ROM BIOS 的固定起始地址开始执行 POST（加电自检）
B 项正确：MBR 大小为 512 字节（位于 0 头 0 柱面 1 扇区），包含 446 字节引导代码、64 字节分区表（DPT）以及 2 字节结束标记 `0x55AA`
C 项错误：MMU 分页机制必须在 CPU 进入保护模式（或长模式）、设置控制寄存器 CR0 的 PG 位（Paging）后才能开启。16 位实模式下仅支持分段寻址（基址 × 16 + 偏移），无法开启 MMU 分页
D 项正确：BIOS 仅在开机初期建立实模式下的简易中断向量表（IVT）。操作系统内核加载后，会建立保护模式下的中断描述符表（IDT），并用 `LIDT` 指令将其装入 CPU，彻底接管系统的中断/异常响应

28. 【考点：操作系统发展历程】操作系统经历了单道批处理、多道批处理、分时与实时系统的演进。下列关于操作系统发展历程及特性的叙述中，正确的是（ ）

A. 单道批处理系统通过引入中断技术实现了 CPU 与 I/O 设备的并行工作，从而完全消除了 CPU 的等待时间
B. 多道程序设计环境下，程序的执行失去了封闭性和可再现性，即相同输入的程序多次运行的结果可能不同或过程不可预知
C. 分时操作系统为了保证高优先级紧急任务的及时响应，核心调度算法采用硬实时抢占调度
D. 实时操作系统（RTOS）的主要设计目标是最大化系统的吞吐量和 CPU 利用率

【答案】： B
【解析】：
A 项错误：单道批处理系统内存中只有一道程序，当该程序等待 I/O 时，CPU 只能空闲等待，无法切换到其他程序，因此不能消除 CPU 的等待时间；多道批处理系统才实现了 CPU 与 I/O 的高度并行
B 项正确：单道环境下程序独占资源，具有封闭性和可再现性。多道环境下多个程序共享资源、并发执行、相互制约，其执行速度取决于系统资源的分配与调度，因而破坏了程序的封闭性（执行结果受外部环境影响）和可再现性
C 项错误：分时系统的核心设计目标是人机交互性与公平响应，主要采用时间片轮转（RR）算法；硬实时抢占调度是实时操作系统的特征
D 项错误：实时系统的核心目标是严格的时间确定性与及时响应能力（可靠性）；而“最大化系统吞吐量和 CPU 利用率”是多道批处理系统的设计目标

29. 【考点：操作系统基本概念与四大特征的关系】并发、共享、虚拟和异步是操作系统的四大基本特征。下列关于这四大特征底层交织作用的说法中，正确的是（ ）

A. 虚拟性（如虚拟内存和虚拟 CPU）以并发和共享为前提；若系统失去了并发性，虚拟性将失去存在意义
B. 互斥共享与同时共享的根本区别在于：同时共享允许多个进程在同一物理时刻真正并行地访问同一物理资源
C. 异步性是指进程以不可预知的速度向前推进，这种不确定性会导致经过严格同步互斥控制的程序每次运行的最终结果不同
D. 只要硬件配备了多核 CPU，即使操作系统内核不支持多线程调度，用户态的多线程程序也能实现物理上的并行执行

【答案】： A
【解析】：
A 项正确：并发和共享是操作系统最基础的两个特征，互为存在条件。虚拟技术（时分复用或空分复用）是将物理实体变为多个逻辑实体，其前提是存在多个进程并发执行并共享资源。若无并发，单进程独占资源，虚拟性就毫无意义
B 项错误：同时共享（如磁盘）是指宏观上“同时”，微观上依然是多个进程交替/分时访问；在单核或单一物理通道上，不可能在同一物理时刻真正“并行”访问
C 项错误：虽然异步性使得进程推进速度不可预知，但通过 OS 提供的同步与互斥机制（如信号量、锁），可以保证程序执行结果的可再现性和正确性，不会导致最终结果不同
D 项错误：如果 OS 内核不支持多线程（例如仅支持用户级线程 ULT，且无内核线程映射），内核只能将整个进程调度到某一个 CPU 核心上，该进程内的多个用户线程无法被分配到不同物理核，无法实现物理并行

30. 【考点：系统调用全流程】当用户程序调用 C 语言库函数 `read()` 从磁盘读取数据时，系统触发系统调用并完成响应。下列关于该全过程微观执行链路的说法中，正确的是（ ）

A. 用户程序执行 `read()` 库函数时，库函数本身运行在内核态，并直接向磁盘控制器发送 I/O 命令
B. 执行陷入指令（Trap/Syscall）引发模式切换时，CPU 硬件隐指令会将当前 PSW、PC 及用户栈指针压入用户栈中保存
C. 系统调用服务例程执行完毕后，CPU 在内核态下执行特权指令（如 `IRET`），该指令会自动恢复 PSW、PC，并将特权级切换回用户态

D. 在系统调用执行期间，若发生缺页异常（Fault），操作系统将认为该用户程序存在严重错误并直接终止（Terminate）该进程

【答案】： C

【解析】：

A 项错误：调用 C 库函数 `read()` 时，程序仍处于用户态；库函数内部将系统调用号和参数装入寄存器后，显式触发陷入指令（如 `syscall` 或 `int 0x80`），才进入内核态

B 项错误：发生特权级切换（用户态 → 内核态）时，CPU 硬件隐指令会自动将上下文（SS, ESP, EFLAGS, CS, EIP）压入内核栈中。用户栈在用户态下可被恶意篡改，内核绝对不会将关键硬件上下文压入用户栈

C 项正确：系统调用处理完毕后，内核执行特权指令 `IRET`（或 `sysret`）。该指令从内核栈中弹出保存的 CS:EIP 和 EFLAGS，同时恢复 SS:ESP，一步完成硬件上下文恢复与特权级降级（内核态 → 用户态）

D 项错误：系统调用过程中（例如内核向用户空间缓冲区拷贝数据）触发缺页属于正常现象，操作系统会暂停当前系统调用，调用页故障处理程序将所需物理页调入内存，随后重新执行引发缺页的访存指令，绝不会终止进程